

Lab 7

PHẦN I: SO SÁNH VÀ PHÂN VÙNG DỮ LIỆU

Bài 1: Window Function với GROUP BY (2.5 điểm)

1. Thử nghiệm với GROUP BY:

SQL

```
SELECT product_name, AVG(price) as avg_price
```

```
FROM products
```

```
GROUP BY product_name;
```

- **Vấn đề:** Khi dùng GROUP BY, dữ liệu bị gộp lại theo tên sản phẩm. Vì mỗi tên sản phẩm thường là duy nhất, AVG(price) lúc này chỉ là giá của chính sản phẩm đó chứ không phải trung bình của cả cửa hàng.

2. Sử dụng Window Function:

SQL

```
SELECT
```

```
    product_name,
```

```
    price,
```

```
    AVG(price) OVER () AS avg_overall_price
```

```
FROM products;
```

3. Quan sát và trả lời:

- **So sánh:** GROUP BY làm mất đi các chi tiết của từng dòng (chỉ còn lại dòng đại diện cho nhóm), trong khi Window Function cho phép ta giữ nguyên tất cả các dòng dữ liệu gốc và vẫn thực hiện được tính toán trên toàn bộ tập dữ liệu.

Bài 2: Phân tích theo danh mục với PARTITION BY (2.5 điểm)

Để làm bài này, bạn đừng quên thêm cột category vào bảng products như yêu cầu chuẩn bị nhé.

Câu lệnh SQL:

SQL

```
SELECT
```

```
    category,
```

product_name,

price,

AVG(price) OVER (PARTITION BY category) AS avg_category_price

FROM products;

- **Giải thích:** PARTITION BY category giúp chia dữ liệu thành từng "cửa sổ" theo loại sản phẩm. Hàm AVG sẽ tính giá trung bình riêng cho từng nhóm đó.

PHẦN II: XẾP HẠNG VÀ TÍNH LŨY KẾ

Bài 3: Xếp hạng sản phẩm (2.5 điểm)

1. Chuẩn bị (Cập nhật giá bằng nhau):

SQL

UPDATE products SET price = 35000 WHERE product_id IN (1, 2);

2. Thực hành hàm xếp hạng:

SQL

SELECT

product_name,

price,

ROW_NUMBER() OVER (ORDER BY price DESC) AS row_num,

RANK() OVER (ORDER BY price DESC) AS rank_num,

DENSE_RANK() OVER (ORDER BY price DESC) AS dense_rank_num

FROM products

ORDER BY price DESC;

3. Quan sát và giải thích sự khác biệt:

- **ROW_NUMBER():** Đánh số thứ tự tăng dần liên tục (1, 2, 3...) bất kể giá có trùng nhau hay không.
- **RANK():** Các sản phẩm cùng giá sẽ có cùng hạng. Tuy nhiên, nó sẽ **nhảy bậc** (ví dụ: 1, 2, 2, 4 - không có hạng 3).
- **DENSE_RANK():** Các sản phẩm cùng giá có cùng hạng, nhưng số hạng tiếp theo sẽ **liên tục** (ví dụ: 1, 2, 2, 3).

Bài 4: Tính tổng lũy kế doanh thu (2.5 điểm)

Bài này yêu cầu bạn dùng CTE để xử lý dữ liệu trước khi tính lũy kế.

Câu lệnh SQL:

SQL

WITH daily_revenue AS (

-- Bước 1: Tính tổng doanh thu theo từng ngày

SELECT

o.order_date,

SUM(oi.quantity * oi.price) AS total_daily_revenue

FROM orders o

JOIN order_items oi ON o.order_id = oi.order_id

GROUP BY o.order_date

)

-- Bước 2: Sử dụng Window Function để tính lũy kế

SELECT

order_date,

total_daily_revenue,

SUM(total_daily_revenue) OVER (ORDER BY order_date) AS running_total_revenue

FROM daily_revenue;

- **Giải thích:** Cú pháp SUM(...) OVER (ORDER BY order_date) sẽ cộng dồn doanh thu từ ngày đầu tiên cho đến ngày hiện tại trên từng dòng.